

1. Objetivo

Utilizar la memoria global del *device* para leer y escribir datos.

2. Ejercicio

Transferir un array de datos entre la memoria del *host* y la memoria del *device* de acuerdo con el esquema mostrado en la Figura 1.1:

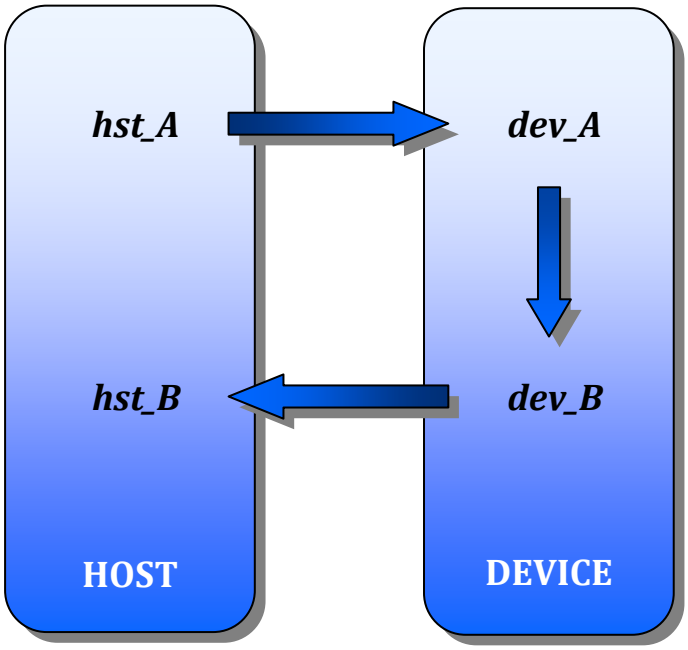


Figura 1.1. Transferencia de datos entre *host* y *device*.

Los arrays `hst_A` y `hst_B` se reservan en la memoria del *host* y los arrays `dev_A` y `dev_B` se reservan en la memoria global del *device*, todos ellos declarados de tipo `float`. Desde el punto de vista del programador es conveniente distinguir entre las variables reservadas en la memoria del *host* y las reservadas en la memoria del *device*. Por ese motivo resulta de gran utilidad añadir un prefijo que permita diferenciales. En nuestro caso hemos utilizado el prefijo `hst_` para las variables del *host* y el prefijo `dev_` para las variables del *device*.

El programa debe mostrar las principales propiedades del dispositivo seleccionado y además cumplir los siguientes requisitos:

» Inicializar el array `hst_A` en el *host* con `N` elementos aleatorios de tipo `float` comprendidos entre 0 y 1.

» Imprimir por pantalla los arrays `hst_A` y `hst_B` utilizando dos cifras decimales con el fin de comprobar que al terminar las transferencias ambos son iguales.

3. Solución propuesta

```
/*
* ARQUITECTURA DE COMPUTADORES
* 2º Grado en Ingenieria Informatica
* Introduccion a la programacion en CUDA
*
* Basico 1: "Transferencia de datos"
* >> Copiar datos GPU <-> CPU
*
*/
////////////////////////////////////////////////////////////////////////
// includes
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <cuda_runtime.h>

////////////////////////////////////////////////////////////////////////
// defines
#define N 16

// declaracion de funciones
// HOST: funcion llamada desde el host y ejecutada en el host
__host__ void propiedades_Device();
// Funcion que imprime las propiedades del device

////////////////////////////////////////////////////////////////////////
// MAIN: rutina principal ejecutada en el host
int main(int argc, char** argv)
{
	// buscando dispositivos
	int deviceCount;
	cudaGetDeviceCount(&deviceCount);
	if (deviceCount == 0)
	{
		printf("!!!!!No se han encontrado dispositivos CUDA!!!!!\n");
		printf("<pulsa [INTRO] para finalizar>");
		getchar();
		return 1;
	}
	else
	{
		propiedades_Device();
	}
	// declaraciones
	float *hst_A, *hst_B;
	float *dev_A, *dev_B;

	// reserva en el host
	hst_A = (float*)malloc(N * sizeof(float));
	hst_B = (float*)malloc(N * sizeof(float));

	// reserva en el device
	cudaMalloc((void**)&dev_A, N * sizeof(float));
	cudaMalloc((void**)&dev_B, N * sizeof(float));

	// incializacion
	srand((int)time(NULL));
	for (int i = 0; i<N; i++)
	{
		hst_A[i] = (float)rand() / RAND_MAX;
```

```
			hst_B[i] = 0;
	}

	// copia de datos
	cudaMemcpy(dev_A, hst_A, N * sizeof(float), cudaMemcpyHostToDevice);
	cudaMemcpy(dev_B, dev_A, N * sizeof(float), cudaMemcpyDeviceToDevice);
	cudaMemcpy(hst_B, dev_B, N * sizeof(float), cudaMemcpyDeviceToHost);

	// muestra de resultados
	printf("ENTRADA (hst_A):\n");
	for (int i = 0; i<N; i++)
		printf("%.2f ", hst_A[i]);
	printf("\n");
	printf("SALIDA (hst_B):\n");
	for (int i = 0; i<N; i++)
		printf("%.2f ", hst_B[i]);
	printf("\n");

	// liberacion de recursos
	cudaFree(dev_A);
	cudaFree(dev_B);

	// salida del programa
	time_t fecha;
	time(&fecha);
	printf("*****************************************************\n");
	printf("Programa ejecutado el: %s\n", ctime(&fecha));
	printf("<pulsa [INTRO] para finalizar>");
	getchar();
	return 0;
}
////////////////////////////////////////////////////////////////////////
// HOST: funcion llamada desde el host y ejecutada en el host
__host__ void propiedades_Device()
{
	int deviceID;
	cudaGetDevice(&deviceID);
	int runtimeVersion;
	cudaRuntimeGetVersion(&runtimeVersion);
	cudaDeviceProp deviceProp;
	cudaGetDeviceProperties(&deviceProp, deviceID);
	// calculo del numero de cores (SP)
	int cudaCores = 0;
	int SM = deviceProp.multiProcessorCount;
	int major = deviceProp.major;
	int minor = deviceProp.minor;
	const char *archName;
	switch (major)
	{
	case 1:
		//TESLA
		archName = "TESLA";
		cudaCores = 8;
		break;
	case 2:
		//FERMI
		archName = "FERMI";
		if (minor == 0)
			cudaCores = 32;
		else
			cudaCores = 48;
		break;
	case 3:
		//KEPLER
		archName = "KEPLER";
		cudaCores = 192;
		break;
	case 5:
		//MAXWELL
		archName = "MAXWELL";
		cudaCores = 128;
		break;
	case 6:
		//PASCAL
		archName = "PASCAL";
		cudaCores = 64;
		break;
	case 7:
		//VOLTA (7.0) TURING (7.5)
		cudaCores = 64;
		if (minor == 0)
```

```
                        archName = "VOLTA";
                else
                        archName = "TURING";
                break;
        case 8:
                //AMPERE
                archName = "AMPERE";
                cudaCores = 64;
                break;
        default:
                //ARQUITECTURA DESCONOCIDA
                archName = "DESCONOCIDA";
                cudaCores = 0;
        }
        // presentacion de propiedades
        printf("****************************************************\n");
        printf("DEVICE %d: %s\n", deviceID, deviceProp.name);
        printf("****************************************************\n");
        printf("> CUDA Toolkit                      \t: %d.%d\n", runtimeVersion / 1000,
(runtimeVersion % 1000) / 10);
        printf("> Arquitectura CUDA                 \t: %s\n", archName);
        printf("> Capacidad de Computo              \t: %d.%d\n", major, minor);
        printf("> No. de MultiProcesadores          \t: %d\n", SM);
        printf("> No. de CUDA Cores (%dx%d)         \t: %d\n", cudaCores, SM, cudaCores*SM);
        printf("> Memoria Global (total)            \t: %zu MiB\n", deviceProp.totalGlobalMem /
(1024 * 1024));
        printf("****************************************************\n");
}
////////////////////////////////////////////////////////////////////////////
```